Social & Media Platform APIs

Week 12 · APIs module · Textbook Chapter 12

Brian C. Keegan, Ph.D.
Associate Professor, Department of Information Science
University of Colorado Boulder

November 2, 4 & 6, 2026

This week at a glance

Last week the data came from governments obliged to publish it. This week it comes from platforms that share it *at their discretion*, and can stop. We read what federal agencies post, every class live coding, from one lab notebook in three parts.

Monday | Lab, part 1: Bluesky

- An open API with no key: who posts, and is it the agency?

Wednesday | Lab, part 2: more of Bluesky

- More posts than one request returns, and an agency's rhythm

Friday | Lab, part 3: YouTube; then your project

- A keyed API for the same agencies' videos; 15 minutes on your portfolio checkpoint

The lab

`labs/week-12/lab-12.ipynb` in the course repository. Each part is added before its day.

One key, for Friday

`YOUTUBE_KEY`, a YouTube Data API key from a **personal** Google account. Set it as an environment variable, close the terminal, open a new one, and start Jupyter from there.

Due Sunday, Nov 8

Chapter 12's **Recommended Exercises**, from its companion notebook, and portfolio checkpoint 2, *Collect and clean your data*. Graded on good-faith attempt to complete rather than getting everything working.

1. Monday · Live-Coding Lab

This week, every class is live coding

Find it | `labs/` in the course repository

1. Open the **labs** folder of `cuinfoscience/INF04617-Fall2026` on GitHub
2. Open `week-12`, click `lab-12.ipynb`, then **Download raw file**
3. Open it in JupyterLab in the `webdata` environment

Use it | one part a day

- I write the code on screen, and you write it in your copy
- Put **your** email in the User-Agent in Step 0
- Today: Part 1, Steps 0–3; no key, no account
- Wednesday and Friday: download the new copy and start at that part's setup step

Solutions

`lab-12-key.ipynb`, beside it: every cell filled in and run. Fall behind? Copy the cell from the key and keep going.

Nothing to submit

The lab is practice. After each step, a **Discuss** box holds questions we'll talk over before the next.

Error flavors

A `KeyError` on a field means the answer wasn't what you expected: print it and look. A request that fails for everyone is *broken*: tell me, or open an issue.

Platforms share at their discretion

Government APIs exist because a law says the data is public. A platform's API exists because the company chooses to offer it, and it can choose again:

- **Reddit** closed self-service API access on November 11, 2025: a new app now needs Reddit's approval.
- **Spotify** has required every new developer app's owner to have Premium since February 11, 2026, and removed top tracks, popularity, and follower counts.
- **Bluesky** is built on an open protocol: its public API answers anyone, with no account or key.

Chapter 12 tells the Reddit and Spotify stories; the lab reads Bluesky, then YouTube.

Social data is people data

A person made almost every post. Read the terms of service; collect the minimum you need; don't quote a post if a search could find its author. This week we read **federal agencies**, and keep private people's posts and handles out of every output.

From the website to the API: one agency's profile

The National Weather Service at bsky.app/profile/nws.noaa.gov, and the same profile from the API:

```
bsky_base = "https://public.api.bsky.app/xrpc"
profile = requests.get(
    f"{bsky_base}/app.bsky.actor.getProfile",
    params={"actor": "nws.noaa.gov"},
    headers=HEADERS).json()
print(profile["handle"], "|", profile["displayName"])
print("followers:", profile["followersCount"],
      "| posts:", profile["postsCount"])
# nws.noaa.gov | National Weather Service
# followers: 63020 | posts: 1394
```

The method's name is part of the address. No key, no login: Bluesky's **AppView** answers reads for anyone.

Is it really the agency?

Two proofs, both in the API's answer:

- The **handle is a domain**, `nws.noaa.gov`: only whoever controls the domain can take it
- A **blue check**: `"verification"` says Bluesky itself verified it

A search for "National Park Service" also finds "Alt National Park Service", with neither. Collect its posts as the agency's, and your study is wrong.

Five agencies in one table

```
agencies = ["noaa.gov", "nws.noaa.gov",
            "climate.noaa.gov", "noaacomms.noaa.gov",
            "federalreserve.gov"]
rows = []
for handle in agencies:
    p = requests.get(
        f"{bsky_base}/app.bsky.actor.getProfile",
        params={"actor": handle}, headers=HEADERS).json()
    rows.append({"handle": p["handle"],
                 "followers": p["followersCount"],
                 "posts": p["postsCount"],
                 "joined": p["createdAt"][:10]})
accounts = pd.DataFrame(rows).sort_values(
    "followers", ascending=False)
```

Handle	Followers	Posts	Joined
nws.noaa.gov	63,020	1,394	2025-01
climate.noaa.gov	46,966	93	2025-01
noaa.gov	41,937	150	2025-01
federalreserve.gov	8,405	1,029	2024-12
noaacomms.noaa.gov	5,620	117	2025-01

These are the federal accounts with a `.gov` handle on November 2; four are NOAA's. NASA and the Census Bureau have none.

Microsprint

Not every official account has a domain: `librarycongress.bsky.social` and `stlouisfed.bsky.social` (FRED's home) are both verified instead.

What an agency posted this week

```
params = {"actor": "nws.noaa.gov", "limit": 25,
          "filter": "posts_no_replies"}
feed = requests.get(
    f"{bsky_base}/app.bsky.feed.getAuthorFeed",
    params=params, headers=HEADERS).json()["feed"]
posts = []
for item in feed:
    if "reason" in item: # a repost
        continue
    post = item["post"]
    posts.append({"time": post["indexedAt"][:16],
                  "likes": post["likeCount"],
                  "reposts": post["repostCount"],
                  "text": post["record"]["text"][:70]})
```

Most reposted, NWS

Reposts	Post
80	Daylight Saving Time ends …
33	Updated Extreme Rainfall …
33	Here's a more detailed look …

Why skip reposts?

A repost carries someone else's post, and they may be a private person. Skipping them keeps every handle in our table an agency's.

The answer ends with a `cursor`. Wednesday: what is it for?

Before Friday: your YouTube key

1. Sign in to the Google Cloud console with a **personal** Google account, and create a project
2. In its **API Library**, enable **YouTube Data API v3**
3. Under **Credentials**, create an **API key**, and restrict it to the YouTube Data API v3
4. Store it with the `webdata` environment, in a terminal (on Windows, the Anaconda Prompt):

```
conda env config vars set YOUTUBE_KEY="your-key" -n webdata
```

Then close that terminal, open a new one, run `conda activate webdata`, and start Jupyter from it.

It's a password

Never paste the key into a notebook cell or a code file. If it leaks, delete it in the console and make a new one.

The quota

10,000 units a day; most reads cost 1 unit, and searches are capped at 100 a day (Google's guide). Friday's code reads channels and videos, not searches.

2. Wednesday · Live-Coding Lab

Today: Part 2, more of Bluesky

- Follow the `cursor` to read more posts than one request returns, as Wikipedia's `continue` did in week 10
- An agency's rhythm: when it posts, and which posts travel
- Download the new `lab-12.ipynb`; start at Part 2's setup step

This part's frames are added before Wednesday's class, after chapter 12's revision.

3. Friday · Live-Coding Lab

Today: Part 3, YouTube's Data API

- A keyed API: `YOUTUBE_KEY`, read with week 11's safe ways
- The same agencies' channels and videos, and the API's daily quota
- Download the new `lab-12.ipynb`; start at Part 3's setup step

This part's frames are added before Friday's class.

Friday's last 15 minutes: portfolio checkpoint 2

Due Sunday | Collect and clean your data

- **Collection code** that runs, makes polite requests (a User-Agent that identifies you, pauses between requests), and saves the raw data
- **A cleaning step**, with its code and output, that produces a tidy dataset
- **A small sample** of the cleaned data in your repository, with a data dictionary
- **The README's Data section**: the source, when you collected it, how many records, and what you kept out of the repository and why

Use the 15 minutes

Run your collection code once, here, where you can ask. A platform this week? Its terms of service decide what goes in your repository.

Graded

Two criteria, one point each, on what's pushed by Sunday night.

Key takeaways

1. A platform's API exists **at the platform's discretion**: Reddit and Spotify have closed much of theirs since 2024. Date what you collect, and say what access you had.
2. **Check who is posting** before you collect: a domain handle or a verified check, not a familiar name.
3. **Social data is people data**: read public institutions' accounts, and keep private people's posts and handles out of what you save and show.
4. **An open protocol** answers anyone; a **keyed API** counts what you use. Both still need a User-Agent.